

WEEK7 Isolation Levels

Transactions — Isolation Levels

Study notes based on the Stanford lecture video on isolation levels (Widom) — the final video in the transactions series. Builds directly on the ACID properties lecture (see prior notes). This video zooms into **Isolation** specifically and shows how real systems trade consistency for performance.

1. Recap — Why Isolation Levels Exist

Recall: multiple clients each submit a sequence of transactions, and **serializability** guarantees the system's behavior is *equivalent to some serial order* of all those transactions, even though execution is actually interleaved.

Client 1: T1 → T2 → ...

Client 2: T9 → T10 → T11 → ...

Serializability gives strong, understandable guarantees — but it has a cost:

- It relies on **locking protocols**, which add **overhead**.
- Locking **reduces concurrency** (clients can block each other waiting for locks).

Because full serializability isn't always worth that cost, the **SQL standard defines weaker isolation levels** that trade consistency guarantees for **higher concurrency and lower overhead**.

The four levels (weakest → strongest)

Level	Strength
Read Uncommitted	Weakest
Read Committed	↑
Repeatable Read	↑
Serializable	Strongest (this is what "isolation" meant in the ACID lecture)

2. Two Important Properties of Isolation Levels

2.1 Isolation level is set **per transaction**

Each transaction can independently choose its own isolation level. A single client can run one transaction at Serializable and the next at Read Committed.

2.2 Isolation level is "in the eye of the beholder"

A transaction's isolation level only constrains **what that transaction itself may observe**. It has **no effect** on any other concurrently running transaction.

So Client A's transaction could run at Repeatable Read while Client B's transaction runs at Read Uncommitted — **simultaneously**, on the same database — and each gets exactly the guarantees (or lack thereof) that *it* requested, independent of the other.

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
-- (the SQL standard's verbose phrase for setting isolation level)

If unspecified, the **SQL standard default is Serializable**.

2.3 Isolation levels are about reads

The isolation level controls **what values a transaction is permitted to read** — specifically, what it might see that was written by other, possibly-uncommitted, concurrent transactions.

3. Core Concept: Dirty Reads

Definition: A data item is **dirty** if it has been written by a transaction that **has not yet committed**.

Why dirty data is dangerous

If a transaction is later **rolled back** (e.g., due to a crash before commit — see Atomicity from the ACID lecture), all of its writes are undone. **Anyone who read a dirty value in the meantime read something that never actually existed in the committed database.**

Example 1 — single dirty value

T1: UPDATE College SET enrollment = enrollment + 1000 WHERE cName = 'Stanford';
[commit]

T2: SELECT AVG(enrollment) FROM College;
[commit]

If T2 reads Stanford's enrollment **before T1 commits**, it sees the +1000 value — but if T1 later fails and rolls back, that +1000 never actually happened. T2's average was computed from a value that **never existed** in the database.

Example 2 — dirty reads in multiple places

T1: UPDATE Student SET GPA = GPA + 0.1 WHERE sizeHS > 1000;
[commit]

T2: SELECT GPA FROM Student WHERE sID = 123;
[commit]

T3: UPDATE Student SET sizeHS = sizeHS + 50 WHERE sID = 234;
[commit]

Two distinct dirty-read possibilities here:

- If T2 reads student 123's GPA **before T1 commits**, that's a dirty read of GPA.
- If something reads student 234's sizeHS **before T3 commits**, that's a dirty read of sizeHS.

Clarification — no such thing as a dirty read *within the same transaction*

If T3 modifies sizeHS and then **later reads its own modification** later in the same transaction, that is **not** a dirty read.

A read is "dirty" **only** when it reads an **uncommitted value written by a different transaction**. Reading your own transaction's uncommitted writes is completely normal and expected.

4. The Four Isolation Levels in Detail

4.1 READ UNCOMMITTED — weakest

Rule: Transactions at this level **may perform dirty reads** — they may read values written by other transactions that haven't committed yet.

Worked example

T1: UPDATE Student SET GPA = GPA + 0.1 WHERE sizeHS > 1000;

T2 (READ UNCOMMITTED):

SELECT AVG(GPA) FROM Student;

If T2's isolation level were the **default (Serializable)**, behavior would be guaranteed equivalent to either:

- T1 then T2 → average reflects **all** GPAs updated, or
- T2 then T1 → average reflects **none** of the GPAs updated.

But with T2 set to READ UNCOMMITTED, the average might be computed **mid-way through T1's updates** — some rows updated, some not. This is **not equivalent to any serial order at all**.

When is this acceptable? When the application genuinely doesn't need precise consistency — e.g., a rough/approximate average is fine, even if it might reflect a change that's later rolled back. In exchange: **higher concurrency, lower overhead, better raw performance**.

4.2 READ COMMITTED — one step stronger

Rule: Transactions may **not** perform dirty reads. They may only read values whose writes have been **committed**. But this still **does not guarantee full (global) serializability**.

Worked example

T1: UPDATE Student SET GPA = GPA + 0.1 WHERE sizeHS > 1000;

T2 (READ COMMITTED):

stmt1: SELECT AVG(GPA) FROM Student; -- reads BEFORE T1

stmt2: SELECT MAX(GPA) FROM Student; -- reads AFTER T1

A legal READ COMMITTED execution: stmt1 runs (and reads) **before** T1's changes, while stmt2 runs (and reads) **after** T1 commits.

Checking against serial orders:

- Not T1 then T2 — because stmt1 saw the *pre*-T1 state.
- Not T2 then T1 — because stmt2 saw the *post*-T1 state.

No serial order matches. Both individual reads were of *committed* data (no dirty reads occurred) — but the overall transaction's behavior is still **not serializable**, because the two reads within T2 are inconsistent with each other.

This phenomenon — reading the *same logical query* twice within one transaction and getting **different answers** because another transaction committed in between — is called a **non-repeatable read**.

4.3 REPEATABLE READ — strongest below Serializable

Rule: No dirty reads (same as Read Committed), **plus:** if a data item is read **multiple times** within the transaction, it **cannot change value** between those reads.

This directly prevents the non-repeatable-read problem from §4.2.

Worked example — repeatable read still isn't fully serializable

T1: stmt1: UPDATE Student SET GPA = GPA + 0.1 ...;
stmt2: UPDATE Student SET sizeHS = sizeHS + 50 WHERE sID = 123;

T2 (REPEATABLE READ):

stmt1: SELECT AVG(GPA) FROM Student;
stmt2: SELECT AVG(sizeHS) FROM Student;

A legal REPEATABLE READ execution: T2's first statement (AVG(GPA)) reads **before** T1 runs; T2's second statement (AVG(sizeHS)) reads **after** T1 commits.

Checking the conditions:

- No dirty reads — both reads are of committed data (before-T1 and after-T1 respectively, both legitimate committed states).
- No value read multiple times — GPA and sizeHS are each read **only once**, so the "can't change between repeated reads" rule isn't even triggered.

So this execution is legal under REPEATABLE READ.

But it's still not serializable:

- Not T1 then T2 — T2's first statement saw the pre-T1 state.
- Not T2 then T1 — T2's second statement saw the post-T1 state.

Key insight: "Repeatable read" only constrains values that are *actually read more than once*. It says nothing about consistency **across different items** read at different times within the same transaction.

The phantom tuple problem

REPEATABLE READ has a specific, important loophole: **insertions are not blocked**, even between two reads of the *same relation*.

Worked example — phantoms via INSERT

T1: INSERT INTO Student (100 new student rows);

T2 (REPEATABLE READ):

stmt1: SELECT AVG(GPA) FROM Student;

stmt2: SELECT MAX(GPA) FROM Student;

REPEATABLE READ **does** allow stmt1 to execute before T1's inserts and stmt2 to execute after them. Why is this legal? Because the *specific rows* read the first time (the original students) **still have the same values** the second time — none of the originally-read tuples changed. The **100 new rows** are simply additional rows that "appeared out of nowhere" during the transaction. These newcomers are called **phantom tuples** (or "phantoms").

If stmt2 had also been AVG(GPA) instead of MAX(GPA), the two averages could come out **different**, even though REPEATABLE READ was honored — because the underlying *set of rows* changed, even though no individual previously-read row's value changed.

Why does this happen? (brief implementation note, not required knowledge)

Repeatable Read is typically implemented by **locking rows once they're read**, preventing modification. But newly **inserted** rows were never locked in the first place (they didn't exist yet), so they're free to appear in a later read of the same relation.

Practical takeaway: If you use REPEATABLE READ, you must know that **inserts by other transactions can still sneak rows into a relation you've already read from**, between two full reads of that relation.

Asymmetry: phantoms via DELETE are blocked

If instead T1 **deletes** 100 rows (instead of inserting them), REPEATABLE READ **does not** allow the "before/after" execution pattern above. Once T2's first read has touched those rows, they are **locked** — T1's delete would be blocked until T2 finishes.

Summary of the asymmetry: Under REPEATABLE READ, a relation **can gain new "phantom" rows** between two reads of it, but it **cannot lose rows** that were already read.

4.4 SERIALIZABLE — strongest (recap)

No dirty reads, no non-repeatable reads, **and** no phantoms. Execution is always equivalent to some full serial order of all transactions. This is the level discussed throughout the ACID-properties lecture.

5. Summary Table — The Definitive Comparison

This table is the heart of the lecture. Memorize it.

Isolation Level	Dirty reads?	Non-repeatable reads?	Phantom tuples?
Read Uncommitted (weakest)	possible	possible	possible
Read Committed	prevented	possible	possible
Repeatable Read	prevented	prevented	possible
Serializable (strongest)	prevented	prevented	prevented

Reading the table **bottom to top**: each level fixes exactly one more problem than the level below it, in this order — dirty reads first, then non-repeatable reads, then phantoms.

6. Read-Only Transactions (a separate, orthogonal flag)

A transaction can also be marked **READ ONLY** — this is a **separate setting from isolation level**, used purely as a **performance hint**.

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ READ ONLY;

- Declares that the transaction **will not perform any modifications** to the database.
 - Lets the system potentially use **lighter-weight protocols** to guarantee the chosen isolation level, since it doesn't need to worry about this transaction's own writes conflicting with anyone else.
 - It's a **hint/optimization**, not itself a consistency guarantee — orthogonal to (and combinable with) any of the four isolation levels.
-

7. Practical Notes

- **The SQL standard default is Serializable.**
 - **Some major systems default to something weaker.** Both **Oracle** and **MySQL** default to **Repeatable Read** rather than Serializable — these vendors assume most applications are willing to trade a bit of consistency for better out-of-the-box performance.
 - Always check **your specific DBMS's default** — don't assume it matches the SQL standard.
-

8. Summary

- **Full serializability has overhead** (locking) and **reduces concurrency**, which is why the SQL standard defines **three weaker isolation levels**: Read Uncommitted, Read Committed, Repeatable Read — alongside the strongest, Serializable.
- Isolation level is **per-transaction** and **"in the eye of the beholder"** — it governs only what that transaction may see, and doesn't constrain any other concurrent transaction.
- The core phenomena that distinguish the levels:
 - **Dirty read** — reading a value written by an uncommitted transaction (note: never "dirty" to read your *own* uncommitted writes).

- o **Non-repeatable read** — reading the same item twice in one transaction and getting two different (but both committed) values, because another transaction committed an update in between.
- o **Phantom tuple** — a *new row* appearing in a relation between two reads of that same relation, due to another transaction's insert (deletes, by contrast, are blocked once read).
- Use the comparison table (§5) as the canonical reference for which phenomena each level allows.
- **READ ONLY** is an orthogonal performance hint, not an isolation level itself.
- **Defaults vary by vendor** — SQL standard says Serializable; Oracle and MySQL actually default to Repeatable Read.

Quick reference

Isolation levels, weakest → strongest:

READ UNCOMMITTED → dirty reads OK, non-repeatable reads OK, phantoms OK

READ COMMITTED → no dirty reads, non-repeatable reads OK, phantoms OK

REPEATABLE READ → no dirty reads, no non-repeatable reads, phantoms OK

SERIALIZABLE → no dirty reads, no non-repeatable reads, no phantoms

SET TRANSACTION ISOLATION LEVEL <level> [READ ONLY];

This concludes the transactions series: transactions solve concurrency control and crash recovery; ACID formalizes the guarantees; isolation levels let applications trade consistency for performance when full serializability isn't necessary.